



СБОРНИК ЗАДАНИЙ И РЕШЕНИЙ

Python — Часть 2: Продвинутый уровень

Строки • Словари • Кортежи • Файлы • Модули • Ошибки • ООП

☆ Лёгкие

★★ Средние

★★★ Сложные

□ **Как работать с этим сборником:** Сначала попробуй решить сам — прикрой решение листом бумаги. Потом сравни. Главное — читай пояснение к каждой строке кода. Тесты в конце каждой темы — для самопроверки.

Тема 8: Строки и работа с текстом

Строки в Python — это объекты с богатым набором методов. Освоив их, ты сможешь обрабатывать любой текст.

Задание 8.1: Паспорт строки

☆☆☆

Задай строку `s = " Привет, Python! "` (с пробелами по краям).

Выведи:

- длину строки (с пробелами)
- строку без пробелов по краям
- строку в ВЕРХНЕМ регистре
- строку в нижнем регистре
- первый и последний символы (после `strip`)

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>s = " Привет, Python! "</code>	Строка с пробелами по краям
<code>print(len(s))</code>	<code>len()</code> считает ВСЕ символы включая пробелы → 20
<code>clean = s.strip()</code>	<code>.strip()</code> убирает пробелы и <code>\n</code> по краям
<code>print(clean)</code>	«Привет, Python!» — без краевых пробелов
<code>print(clean.upper())</code>	<code>.upper()</code> — все буквы заглавные
<code>print(clean.lower())</code>	<code>.lower()</code> — все буквы строчные
<code>print(clean[0])</code>	Индекс 0 — первый символ: «П»
<code>print(clean[-1])</code>	Индекс -1 — последний символ: «!»

► Результат:

```
20
Привет, Python!
ПРИВЕТ, PYTHON!
привет, python!
П
!
```

⊖ Частая ошибка:

✗ `print(s[0])` # выведет пробел!

✓ `print(s.strip()[0])` # сначала `strip`, потом индекс

Индексация работает на исходной строке. Если нужен первый «настоящий» символ — сначала применяй `strip()`.

Задание 8.2: Обработка CSV-строки

★★☆

Дана строка с данными: `data = "Иванов,Иван,17,Москва,9А"`
Извлеки каждое поле через `split()`.
Выведи красиво:
Фамилия: Иванов
Имя: Иван
...и т.д.
Потом собери обратно через `join()` с разделителем `" | "`.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>data = "Иванов,Иван,17,Москва,9А"</code>	<i>Входная строка — поля через запятую</i>
<code>parts = data.split(",")</code>	<i>.split(разделитель) → список строк</i>
<code># parts = ['Иванов', 'Иван', '17', 'Москва', '9А']</code>	
<code>labels = ["Фамилия", "Имя", "Возраст", "Город", "Класс"]</code>	<i>Метки полей — в том же порядке</i>
<code>for label, value in zip(labels, parts):</code>	<i>zip() соединяет два списка в пары</i>
<code> print(f"{label}: {value}")</code>	<i>f-строка выводит каждую пару</i>
<code>rejoined = " ".join(parts)</code>	<i>.join(список) — склеивает через разделитель</i>
<code>print(rejoined)</code>	<i>Иванов Иван 17 Москва 9А</i>

► Результат:

```
Фамилия: Иванов
Имя: Иван
Возраст: 17
Город: Москва
Класс: 9А
Иванов | Иван | 17 | Москва | 9А
```

Задание 8.3: Форматирование таблицы

★★☆

Создай список студентов:
`[("Алёша", 95), ("Маша", 88), ("Дима", 72), ("Катя", 91)]`
Выведи красиво отформатированную таблицу с помощью f-строк.
Столбцы: имя (ширина 10), оценка (ширина 5).
В конце — средний балл.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>students = [</code>	Список кортежей (имя, оценка)
<code> ("Алёша", 95), ("Маша", 88),</code>	
<code> ("Дима", 72), ("Катя", 91),</code>	
<code>]</code>	
<code>print(f"{'Имя':<10} {'Балл':>5}")</code>	:<10 — выравнивание влево, ширина 10
<code>print("-" * 16)</code>	Разделитель
<code>total = 0</code>	Накопитель для суммы баллов
<code>for name, score in students:</code>	Распаковка кортежа прямо в цикле
<code> print(f"{'name':<10} {'score':>5}")</code>	:>5 — выравнивание вправо, ширина 5
<code> total += score</code>	+= прибавляет к накопителю
<code>avg = total / len(students)</code>	Считаем среднее
<code>print("-" * 16)</code>	
<code>print(f"{'Среднее':<10} {'avg':>5.1f}")</code>	:.1f — дробное число, 1 знак после точки

► Результат:

Имя	Балл
Алёша	95
Маша	88
Дима	72
Катя	91
Среднее	86.5

Дана строка с предложением. Напиши программу, которая:

1. Подсчитает количество слов
2. Найдёт самое длинное слово
3. Подсчитает количество каждой гласной (а,е,и,о,у,ы,э,ю,я)
4. Проверит, является ли строка палиндромом (игнорируй пробелы и регистр)

Тестовая строка: "А роза упала на лапу Азора"

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>text = "А роза упала на лапу Азора"</code>	<i>Известный русский палиндром</i>
<code># 1. Количество слов</code>	
<code>words = text.split()</code>	<i>.split() без аргумента делит по пробелам</i>
<code>print(f"Слов: {len(words)}")</code>	<i>len списка = количество слов</i>
<code># 2. Самое длинное слово</code>	
<code>longest = max(words, key=len)</code>	<i>max с key=len — ищет максимум по длине</i>
<code>print(f"Самое длинное: {longest}")</code>	
<code># 3. Частота гласных</code>	
<code>vowels = "аеиоуыэюяАЕИОУЫЭЮЯ"</code>	<i>Строка со всеми гласными</i>
<code>for v in "аеиоуыэюя":</code>	<i>Перебираем только строчные</i>
<code> cnt = text.lower().count(v)</code>	<i>.lower() для единообразия, .count() считает</i>
<code> if cnt > 0:</code>	<i>Выводим только те, что встречаются</i>
<code> print(f" '{v}': {cnt}")</code>	
<code># 4. Палиндром</code>	
<code>clean = text.lower().replace(" ", "")</code>	<i>.replace убирает все пробелы</i>
<code>is_pal = clean == clean[::-1]</code>	<i>[::-1] — строка задом наперёд</i>
<code>print(f"Палиндром: {is_pal}")</code>	<i>True если читается одинаково</i>

► Результат:

```
Слов: 6
Самое длинное: упала
'a': 7
'o': 2
'y': 2
Палиндром: True
```

□ Тест: Тема 8

? Вопрос: Что вернёт "hello".upper()?

А) Hello

Б) HELLO ← *правильный ответ*

В) hello

Г) Ошибка

? Вопрос: Что вернёт "a,b,c".split(",")?

А) "abc"

Б) ['a','b','c'] ← *правильный ответ*

В) ('a','b','c')

Г) {'a','b','c'}

? Вопрос: Как вставить переменную name в строку питонически?

А) "Привет, "+name

Б) "Привет, %s" % name

В) f"Привет, {name}" ← *правильный ответ*

Г) format(name)

Тема 9: Словари (dict)

Словарь хранит данные как пары «ключ: значение». Позволяет находить данные мгновенно по ключу.

Задание 9.1: Моя визитка

☆☆☆

Создай словарь person с полями:
name, age, city, hobby, is_student
Выведи каждую пару «ключ → значение» в цикле.
Потом выведи только ключи и только значения отдельно.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

person = {	Создаём словарь
"name": "Дима",	Ключ: строка, значение: строка
"age": 16,	Значение может быть любого типа
"city": "Казань",	
"hobby": "программирование",	
"is_student": True	bool как значение — ок
}	
# Перебор пар	
for key, value in person.items():	.items() возвращает пары (ключ, значение)
print(f"{key:12} → {value}")	key:12 — выравнивание ключа шириной 12
print(list(person.keys()))	.keys() — все ключи
print(list(person.values()))	.values() — все значения

► Результат:

```
name      → Дима
age       → 16
city      → Казань
hobby     → программирование
is_student → True
['name', 'age', 'city', 'hobby', 'is_student']
['Дима', 16, 'Казань', 'программирование', True]
```

⊖ Частая ошибка:

✗ `print(person["phone"])` # `KeyError!`

✓ `print(person.get("phone", "не указан"))`

Обращение к несуществующему ключу через `[]` вызывает `KeyError`. Используй `.get(ключ, значение_по_умолчанию)` — безопасный вариант.

Задание 9.2: Частотный анализ

★★★

Дана строка `text = "банан апельсин банан груша банан апельсин"`.
Подсчитай сколько раз встречается каждое слово.
Используй словарь как счётчик.
Выведи результат отсортированный по убыванию частоты.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>text = "банан апельсин банан груша банан апельсин"</code>	
<code>words = text.split()</code>	Разбиваем на список слов
<code>freq = {}</code>	Пустой словарь-счётчик
<code>for word in words:</code>	Перебираем каждое слово
<code> freq[word] = freq.get(word, 0) + 1</code>	<code>.get(word, 0)</code> : если нет — 0, иначе текущее
<code># Сортировка по значению (по убыванию)</code>	
<code>sorted_freq = sorted(</code>	<code>sorted()</code> возвращает новый список
<code> freq.items(),</code>	<code>.items()</code> — пары (слово, частота)
<code> key=lambda item: item[1],</code>	<code>key</code> — по какому признаку сортировать
<code> reverse=True</code>	<code>reverse=True</code> — по убыванию
<code>)</code>	
<code>for word, count in sorted_freq:</code>	Распаковываем каждую пару
<code> print(f"{word:12}: {count} раз(a)")</code>	

► Результат:

```
банан      : 3 раз(a)
апельсин   : 2 раз(a)
груша      : 1 раз(a)
```

□ **Альтернатива — Counter:** `from collections import Counter; freq = Counter(words)` делает то же самое в одну строку. Но знать ручной способ тоже важно!

Задание 9.3: Записная книжка

★★★

Создай программу-записную книжку на словаре.
Операции:
— добавить контакт (имя → номер)
— найти номер по имени
— удалить контакт
— показать всех
Сделай 4 функции и протестируй их.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

phonebook = {}	Глобальный словарь — наша книжка
def add(name, phone):	Функция добавления
phonebook[name] = phone	Ключ — имя, значение — телефон
print(f"Добавлен: {name}")	
def find(name):	Поиск по имени
result = phonebook.get(name)	.get() вернёт None если не найдено
if result:	None — это False, строка — True
print(f"{name}: {result}")	
else:	
print(f"{name} не найден")	
def remove(name):	Удаление контакта
if name in phonebook:	in проверяет наличие ключа
del phonebook[name]	del удаляет пару из словаря
print(f"Удалён: {name}")	
else:	
print("Контакт не найден")	
def show_all():	Показать всех
if not phonebook:	Пустой словарь — False
print("Книжка пуста")	
return	Выходим из функции
for name, phone in sorted(phonebook.items()):	sorted() — алфавитный порядок
print(f" {name:15} {phone}")	
add("Маша", "+7-916-111-22-33")	Тестируем
add("Алёша", "+7-925-444-55-66")	
find("Маша")	
show_all()	
remove("Маша")	
show_all()	

► Результат:

```

Добавлен: Маша
Добавлен: Алёша
Маша: +7-916-111-22-33
    Алёша          +7-925-444-55-66
    Маша           +7-916-111-22-33
Удалён: Маша
    Алёша          +7-925-444-55-66

```

Задание 9.4: Вложенные словари: школа

★★★

Создай структуру данных — словарь школы:
`school = {"9А": {"teacher": ..., "students": [...]}, ...}`

Минимум 2 класса, по 3 студента с оценками.

Напиши функции:

— `average_class(class_name)` — средний балл класса

— `top_student()` — лучший ученик всей школы

— `class_report()` — отчёт по всем классам

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>school = {</code>	
<code> "9А": {</code>	Ключ первого уровня — название класса
<code> "teacher": "Иванова",</code>	
<code> "students": [</code>	Список студентов — значение ключа
<code> {"name": "Алёша", "score": 95},</code>	Каждый студент — тоже словарь
<code> {"name": "Маша", "score": 88},</code>	
<code> {"name": "Дима", "score": 72},</code>	
<code>]</code>	
<code> },</code>	
<code> "9Б": {</code>	
<code> "teacher": "Петров",</code>	
<code> "students": [</code>	
<code> {"name": "Катя", "score": 91},</code>	
<code> {"name": "Вова", "score": 84},</code>	
<code> {"name": "Света", "score": 78},</code>	
<code>]</code>	
<code> }</code>	
<code>}</code>	
<code>def average_class(cls):</code>	Средний балл класса
<code> scores = [s["score"] for s in school[cls]["students"]]</code>	List comprehension: берём score каждого
<code> return sum(scores) / len(scores)</code>	Среднее = сумма / количество
<code>def top_student():</code>	Лучший по всей школе
<code> best = None</code>	Начинаем с None
<code> for cls in school:</code>	Перебираем классы
<code> for s in school[cls]["students"]:</code>	Перебираем студентов класса
<code> if best is None or s["score"] > best["score"]:</code>	Сравниваем с текущим лучшим
<code> best = s</code>	Обновляем лучшего
<code> return best</code>	
<code>for cls in school:</code>	Выводим отчёт
<code> avg = average_class(cls)</code>	
<code> print(f"{cls}: средний балл {avg:.1f}")</code>	:.1f — 1 знак после запятой
<code>top = top_student()</code>	
<code>print(f"Лучший: {top['name']} ({top['score']})")</code>	

► Результат:

9А: средний балл 85.0

9Б: средний балл 84.3
Лучший: Алёша (95)

□ Тест: Тема 9

? Вопрос: Что вернёт `d.get("key", 0)` если ключа нет?

А) `None`

Б) `KeyError`

В) `0` ← *правильный ответ*

Г) `False`

? Вопрос: Как удалить ключ из словаря?

А) `d.remove(key)`

Б) `d.delete(key)`

В) `del d[key]` ← *правильный ответ*

Г) `d.pop_key(key)`

? Вопрос: Какой метод возвращает все пары (ключ, значение)?

А) `.keys()`

Б) `.values()`

В) `.items()` ← *правильный ответ*

Г) `.pairs()`

Тема 10: Кортежи и множества

Кортеж — неизменяемый список. Множество — коллекция уникальных элементов без порядка.

Задание 10.1: Координаты точек

☆☆☆

Создай три точки как кортежи: A=(1,2), B=(4,6), C=(-3,0).
Выведи координаты каждой точки через распаковку.
Посчитай расстояние от A до B по формуле:
 $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$
Используй `import math`.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>import math</code>	Модуль <code>math</code> нужен для <code>sqrt</code>
<code>A = (1, 2)</code>	Кортеж — скобки, элементы через запятую
<code>B = (4, 6)</code>	Кортеж нельзя изменить после создания
<code>C = (-3, 0)</code>	
<code>for name, point in [("A", A), ("B", B), ("C", C)]:</code>	Перебираем список кортежей
<code> x, y = point</code>	Распаковка: <code>x</code> и <code>y</code> из кортежа
<code> print(f"Точка {name}: x={x}, y={y}")</code>	
<code># Расстояние A→B</code>	
<code>x1, y1 = A</code>	Распаковка координат A
<code>x2, y2 = B</code>	Распаковка координат B
<code>dist = math.sqrt((x2-x1)**2 + (y2-y1)**2)</code>	Формула Евклидова расстояния
<code>print(f"Расстояние A→B: {dist:.2f}")</code>	<code>:.2f</code> — 2 знака после точки

► Результат:

Точка A: x=1, y=2
Точка B: x=4, y=6
Точка C: x=-3, y=0
Расстояние A→B: 5.00

Задание 10.2: Уникальные посетители

☆☆☆

Есть два списка ID посетителей сайта за два дня:
`day1 = [1,2,3,4,5,2,3]`
`day2 = [3,4,5,6,7,8,3]`
Используя множества, найди:
— уникальных посетителей за каждый день
— кто пришёл в оба дня (пересечение)
— кто пришёл хотя бы один раз (объединение)
— кто пришёл только в первый день (разность)

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>day1 = [1,2,3,4,5,2,3]</code>	Список с дубликатами
<code>day2 = [3,4,5,6,7,8,3]</code>	
<code>set1 = set(day1)</code>	<code>set()</code> убирает дубликаты → {1,2,3,4,5}
<code>set2 = set(day2)</code>	→ {3,4,5,6,7,8}
<code>print(f"День 1 уникальных: {len(set1)}")</code>	Уникальных 5
<code>print(f"День 2 уникальных: {len(set2)}")</code>	Уникальных 6
<code>both = set1 & set2</code>	& — пересечение: кто был в оба дня
<code>any_day = set1 set2</code>	— объединение: все уникальные
<code>only1 = set1 - set2</code>	- — разность: только в set1
<code>print(f"В оба дня: {sorted(both)}")</code>	<code>sorted</code> для красивого вывода
<code>print(f"Всего: {sorted(any_day)}")</code>	
<code>print(f"Только д1: {sorted(only1)}")</code>	

► Результат:

```
День 1 уникальных: 5
День 2 уникальных: 6
В оба дня: [3, 4, 5]
Всего:      [1, 2, 3, 4, 5, 6, 7, 8]
Только д1: [1, 2]
```

□ Тест: Тема 10

? Вопрос: Можно ли изменить элемент кортежа `t = (1,2,3)`?

А) Да через `t[0]=5`

Б) Нет, кортеж неизменяем ← *правильный ответ*

В) Да через `t.set(0,5)`

Г) Зависит от типа

? Вопрос: Что выведет: `print(len({1,2,2,3,3,3}))`?

А) 6

Б) 3 ← *правильный ответ*

В) 1

Г) Ошибка

Тема 11: Работа с файлами

Файлы позволяют программе сохранять данные между запусками. Используй `with` — он всегда закрывает файл.

Задание 11.1: Дневник

☆☆☆

Напиши программу-дневник:

- При каждом запуске показывай все предыдущие записи (если файл есть)
 - Попроси ввести новую запись
 - Добавь её в файл `diary.txt` с новой строки
- Используй режим `'a'` (дозапись).

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>import os</code>	<i>os нужен для проверки существования файла</i>
<code>FILENAME = "diary.txt"</code>	<i>Имя файла — константа (заглавные буквы)</i>
<code># Показываем старые записи</code>	
<code>if os.path.exists(FILENAME):</code>	<i>Проверяем что файл уже существует</i>
<code> print("=== Предыдущие записи ===")</code>	
<code> with open(FILENAME, "r",</code>	<i>Открываем для чтения</i>
<code> encoding="utf-8") as f:</code>	<i>encoding — обязательно для русского</i>
<code> for line in f:</code>	<i>Читаем построчно</i>
<code> print(line.strip())</code>	<i>.strip() убирает \n в конце строки</i>
<code>else:</code>	
<code> print("Дневник пуст. Начни писать!")</code>	
<code># Добавляем новую запись</code>	
<code>entry = input("\nНовая запись: ")</code>	<i>\n — пустая строка перед вопросом</i>
<code>with open(FILENAME, "a",</code>	<i>Режим 'a' — дозапись (не затирает!)</i>
<code> encoding="utf-8") as f:</code>	
<code> f.write(entry + "\n")</code>	<i>\n — перенос строки после записи</i>
<code>print("Сохранено!")</code>	

► Результат:

=== Предыдущие записи ===
Сегодня начал учить Python!
Написал первую программу.

Новая запись: Научился работать с файлами
Сохранено!

⊖ Частая ошибка:

✗ `open("file.txt", "w")` # при каждом запуске

✓ `open("file.txt", "a")` # дозапись

Режим 'w' каждый раз создаёт файл заново, стирая старые данные. Режим 'a' добавляет в конец.

Задание 11.2: Анализ текстового файла

★★☆

Создай файл words.txt с 10 любыми словами (каждое на новой строке).

Напиши программу, которая читает файл и выводит:

- количество строк
- все слова в верхнем регистре
- слова длиннее 5 букв
- слова в алфавитном порядке

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

# Сначала создаём тестовый файл	
words_to_write = ["программирование", "питон", "алгоритм", "цикл", "список", "функция", "класс", "объект", "переменная", "условие"]	
with open("words.txt", "w", encoding="utf-8") as f:	Создаём файл
f.write("\n".join(words_to_write))	.join() склеивает список через \n
# Читаем и анализируем	
with open("words.txt", "r", encoding="utf-8") as f:	Открываем для чтения
words = [line.strip() for line in f]	List comprehension: читаем и сразу strip
print(f"Строк: {len(words)}")	
print("ВЕРХНИЙ:", [w.upper() for w in words])	List comprehension с методом
long = [w for w in words if len(w) > 5]	Фильтр по длине
print(f"Длинные: {long}")	
print("Алфавит:", sorted(words))	sorted() возвращает новый список

► Результат:

Строк: 10

ВЕРХНИЙ: ['ПРОГРАММИРОВАНИЕ', 'ПИТОН', ...]

Длинные: ['программирование', 'алгоритм', 'функция', 'переменная',
'условие']

Алфавит: ['алгоритм', 'класс', 'объект', ...]

□ Тест: Тема 11

? Вопрос: Какой режим открытия файла добавляет данные в конец?

А) 'w'

Б) 'r'

В) 'a' ← *правильный ответ*

Г) 'x'

? Вопрос: Зачем нужен with при работе с файлами?

А) Для скорости

Б) Автоматически закрывает файл ← *правильный ответ*

В) Позволяет читать бинарные файлы

Г) Только для записи

Тема 12: Модули и библиотеки

Модули — готовые наборы функций. Подключай через `import` и пользуйся чужим трудом!

Задание 12.1: Математическая шкатулка

☆☆☆

- Используя модуль `math`, вычисли и выведи:
- $\sin(90^\circ)$, $\cos(0^\circ)$, $\tan(45^\circ)$ (переведи в радианы!)
 - $\sqrt{2}$, $\sqrt[3]{8}$ (кубический корень через степень)
 - $\log_{10}(1000)$, $\log_2(64)$
 - число e (основание натурального логарифма)

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>import math</code>	Подключаем стандартный модуль математики
<code># Тригонометрия — аргументы в радианах!</code>	
<code>print(math.sin(math.pi / 2))</code>	$\sin(90^\circ)$: $\pi/2$ радиан = 90°
<code>print(math.cos(0))</code>	$\cos(0^\circ) = 1.0$
<code>print(math.tan(math.pi / 4))</code>	$\tan(45^\circ)$: $\pi/4$ радиан = 45°
<code># Корни</code>	
<code>print(math.sqrt(2))</code>	<code>math.sqrt</code> — квадратный корень
<code>print(8 ** (1/3))</code>	Кубический корень = степень $1/3$
<code># Логарифмы</code>	
<code>print(math.log10(1000))</code>	$\log$ по основанию 10: $\log_{10}(1000)=3$
<code>print(math.log2(64))</code>	$\log$ по основанию 2: $\log_2(64)=6$
<code>print(math.e)</code>	Число Эйлера ≈ 2.718

► Результат:

```
1.0
1.0
0.9999... (≈1.0)
1.4142135623730951
2.0
3.0
6.0
2.718281828459045
```

Задание 12.2: Генератор паролей

☆☆☆

Используя модуль `random` и `string`, напиши генератор паролей.
 Функция `generate_password(length, use_digits, use_special)` должна:

- принимать длину и флаги типов символов
- возвращать случайный пароль
- гарантировать хотя бы один символ каждого включённого типа

Сгенерируй 5 паролей разных типов.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>import random</code>	Для случайного выбора
<code>import string</code>	Содержит готовые наборы символов
<code>def generate_password(length=12,</code>	Параметры по умолчанию
<code> use_digits=True,</code>	
<code> use_special=True):</code>	
<code> chars = string.ascii_letters</code>	a-z + A-Z — всегда включаем
<code> guaranteed = []</code>	Список гарантированных символов
<code> if use_digits:</code>	Если нужны цифры
<code> chars += string.digits</code>	Добавляем '0123456789' к набору
<code> guaranteed.append(</code>	
<code> random.choice(string.digits))</code>	Гарантируем минимум одну цифру
<code> if use_special:</code>	Если нужны спецсимволы
<code> chars += string.punctuation</code>	Добавляем !@#%\$%...
<code> guaranteed.append(</code>	
<code> random.choice(string.punctuation))</code>	
<code> remaining = length - len(guaranteed)</code>	Сколько символов ещё нужно
<code> pool = [random.choice(chars)</code>	Добираем из всего набора
<code> for _ in range(remaining)]</code>	_ — переменная-заглушка
<code> password = guaranteed + pool</code>	Объединяем гарантированные и остальные
<code> random.shuffle(password)</code>	.shuffle() перемешивает список
<code> return "".join(password)</code>	.join() собирает список в строку
<code>for i in range(5):</code>	Генерируем 5 паролей
<code> pwd = generate_password(12)</code>	
<code> print(f"Пароль {i+1}: {pwd}")</code>	

► Результат:

Пароль 1: aK3!mPx9@nLq
 Пароль 2: R#7wZb2&yTmN
 ... (случайные каждый раз)

□ Тест: Тема 12

? Вопрос: Как правильно использовать только функцию sqrt из модуля math?

А) import math.sqrt

Б) from math import sqrt ← *правильный ответ*

В) include math

Г) using math.sqrt

? Вопрос: Что делает random.shuffle(lst)?

А) Сортирует список

Б) Перемешивает список на месте ← *правильный ответ*

В) Возвращает перемешанную копию

Г) Выбирает случайный элемент

Тема 13: Обработка ошибок (try/except)

Ошибки — часть программирования. Научись их перехватывать и обрабатывать красиво.

Задание 13.1: Безопасный ввод числа

★★★

Напиши функцию `safe_int(prompt)`, которая:

- запрашивает ввод с заданным приглашением
- возвращает целое число если ввод корректный
- повторяет запрос если ввод некорректный
- продолжает спрашивать пока не получит число

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>def safe_int(prompt):</code>	
<code>while True:</code>	<i>Бесконечный цикл — выйдем через return</i>
<code>try:</code>	<i>Пробуем выполнить код</i>
<code>value = int(input(prompt))</code>	<i>Может вызвать ValueError если не число</i>
<code>return value</code>	<i>Если дошли сюда — число корректное</i>
<code>except ValueError:</code>	<i>Перехватываем ошибку преобразования</i>
<code>print("Это не число! Попробуй снова.")</code>	<i>Сообщаем и идём на следующий виток</i>
<code># Использование</code>	
<code>age = safe_int("Введи возраст: ")</code>	<i>Будет спрашивать пока не получит число</i>
<code>print(f"Тебе {age} лет")</code>	<i>Гарантированно целое число</i>

► Результат:

```
Введи возраст: abc
Это не число! Попробуй снова.
Введи возраст: 3.5
Это не число! Попробуй снова.
Введи возраст: 16
Тебе 16 лет
```

Задание 13.2: Калькулятор с защитой

★★★

Напиши калькулятор, который:

- принимает два числа и операцию (+, -, *, /)
- обрабатывает все возможные ошибки:
 - деление на ноль
 - некорректный ввод числа
 - неизвестная операция
- использует несколько блоков `except`

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>def calculate(a, b, op):</code>	<i>Функция вычисления</i>
<code> if op == "+": return a + b</code>	<i>Проверяем операцию</i>
<code> elif op == "-": return a - b</code>	
<code> elif op == "*": return a * b</code>	
<code> elif op == "/":</code>	<i>Деление — особый случай</i>
<code> if b == 0:</code>	<i>Проверяем делитель до деления</i>
<code> raise ZeroDivisionError</code>	<i>raise — намеренно вызываем ошибку</i>
<code> return a / b</code>	
<code> else:</code>	
<code> raise ValueError(f"Неизвестная операция: {op}")</code>	<i>ValueError с описанием</i>
<code>try:</code>	<i>Основной блок</i>
<code> a = float(input("Число 1: "))</code>	<i>float — разрешаем дробные</i>
<code> b = float(input("Число 2: "))</code>	
<code> op = input("Операция (+,-,*,/): ")</code>	
<code> result = calculate(a, b, op)</code>	<i>Вызываем функцию</i>
<code> print(f"Результат: {result}")</code>	
<code>except ValueError as e:</code>	<i>as e — сохраняем ошибку в переменную</i>
<code> print(f"Ошибка ввода: {e}")</code>	<i>e содержит описание ошибки</i>
<code>except ZeroDivisionError:</code>	<i>Отдельный except для деления на 0</i>
<code> print("Ошибка: деление на ноль!")</code>	

► **Результат:**

```
Число 1: 10
Число 2: 0
Операция (+,-,*,/): /
Ошибка: деление на ноль!
```

□ Тест: Тема 13

? Вопрос: Какой блок выполняется в любом случае — и при ошибке, и без?

А) except

Б) else

В) finally ← *правильный ответ*

Г) always

? Вопрос: Как получить текст исключения в переменную e?

А) except Error e:

Б) catch Exception as e:

В) except Exception as e: ← *правильный ответ*

Г) error = try()

Тема 14: Объектно-ориентированное программирование

Классы позволяют объединять данные и методы работы с ними в одну сущность.

Задание 14.1: Класс Rectangle

☆☆☆

Создай класс Rectangle (прямоугольник) с:

- атрибутами width и height
- методом area() — площадь
- методом perimeter() — периметр
- методом is_square() — является ли квадратом
- методом __str__ — красивый вывод

Создай 3 объекта и протестируй все методы.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

class Rectangle:	Объявляем класс
def __init__(self, width, height):	__init__ — конструктор, вызывается при создании
self.width = width	self.width — атрибут объекта
self.height = height	self — ссылка на сам объект
def area(self):	Метод — функция внутри класса
return self.width * self.height	self.width — обращаемся к атрибуту
def perimeter(self):	
return 2 * (self.width + self.height)	Формула периметра
def is_square(self):	
return self.width == self.height	Квадрат — если стороны равны
def __str__(self):	__str__ вызывается при print(объект)
return f"Прямоугольник {self.width}x{self.height}"	
r1 = Rectangle(4, 6)	Создаём объект — вызывается __init__
r2 = Rectangle(5, 5)	Квадрат — стороны равны
r3 = Rectangle(10, 3)	
for r in [r1, r2, r3]:	Тестируем все три
print(r)	Вызовет __str__
print(f" Площадь: {r.area()}")	Вызов метода
print(f" Периметр: {r.perimeter()}")	
print(f" Квадрат? {r.is_square()}")	

► **Результат:**

```
Прямоугольник 4x6
  Площадь: 24
  Периметр: 20
  Квадрат? False
Прямоугольник 5x5
  Площадь: 25
  Периметр: 20
  Квадрат? True
...
```

Задание 14.2: Наследование: зоопарк

★★★

Создай иерархию классов:

- Animal: имя, возраст, метод info(), метод speak()
- Dog(Animal): порода, speak() → «Гав!», метод fetch()
- Cat(Animal): домашняя/уличная, speak() → «Мяу!», метод purr()
- Bird(Animal): может ли летать, speak() → «Чирик!»

Создай по 2 объекта каждого типа, помести в список и вызови speak() у всех.

✓ **РЕШЕНИЕ С ПОЯСНЕНИЯМИ**

class Animal:	Базовый класс
def __init__(self, name, age):	Общие атрибуты всех животных
self.name = name	
self.age = age	
def info(self):	Общий метод
print(f"{self.name}, возраст: {self.age}")	
def speak(self):	Базовая версия — будет переопределена
print(f"{self.name}: ...")	
class Dog(Animal):	Dog наследует Animal — пишем в скобках
def __init__(self, name, age, breed):	Добавляем атрибут породы
super().__init__(name, age)	super() вызывает __init__ родителя
self.breed = breed	Добавляем свой атрибут
def speak(self):	Переопределяем метод родителя
print(f"{self.name}: Гав!")	
def fetch(self):	Уникальный метод только для Dog
print(f"{self.name} принёс мячик!")	
class Cat(Animal):	
def __init__(self, name, age, indoor):	indoor — домашняя или нет
super().__init__(name, age)	
self.indoor = indoor	
def speak(self):	

<code>print(f"{self.name}: Мяу!")</code>	
<code>def purr(self):</code>	
<code>print(f"{self.name}: Муррррр...")</code>	
<code>animals = [</code>	<i>Список разных животных</i>
<code> Dog("Шарик", 3, "Дворняга"),</code>	
<code> Dog("Рекс", 5, "Овчарка"),</code>	
<code> Cat("Мурка", 2, True),</code>	
<code> Cat("Барсик", 4, False),</code>	
<code>]</code>	
<code>for animal in animals:</code>	<i>Полиморфизм — один вызов, разное поведение</i>
<code> animal.speak()</code>	<i>Python сам найдёт нужный speak()</i>

► **Результат:**

Шарик: Гав!
Рекс: Гав!
Мурка: Мяу!
Барсик: Мяу!

Задание 14.3: Класс Stack с историей

★★★

Создай класс Stack с методами:

- `push(item)` — добавить
- `pop()` — убрать с вершины
- `peek()` — посмотреть вершину
- `is_empty()` — пустой?
- `__len__` — `len(stack)`
- `__str__` — красивый вывод

Добавь историю операций: каждая операция записывается.

Метод `history()` выводит все операции с результатами.

class Stack:	
def __init__(self):	
self._data = []	<i>_ перед именем — «защищённый» атрибут</i>
self._history = []	<i>Список для хранения истории</i>
def push(self, item):	
self._data.append(item)	<i>Добавляем на вершину</i>
self._history.append(f"push({item})")	<i>Записываем в историю</i>
def pop(self):	
if self.is_empty():	<i>Защита от пустого стека</i>
self._history.append("pop() — ОШИБКА: стек пуст")	<i>Записываем даже ошибки</i>
raise IndexError("Стек пуст")	<i>raise — вызываем исключение</i>
item = self._data.pop()	<i>.pop() убирает и возвращает последний</i>
self._history.append(f"pop() → {item}")	
return item	
def peek(self):	
return self._data[-1] if self._data else None	<i>Тернарный оператор: x if cond else y</i>
def is_empty(self):	
return len(self._data) == 0	
def __len__(self):	<i>__len__ вызывается при len(объект)</i>
return len(self._data)	
def __str__(self):	
return f"Stack{self._data}"	
def history(self):	
print("История операций:")	
for i, op in enumerate(self._history, 1):	<i>enumerate даёт номер с 1</i>
print(f" {i}. {op}")	
s = Stack()	<i>Создаём объект</i>
s.push(10)	
s.push(20)	
s.push(30)	
print(s.pop())	<i>30</i>
print(f"Вершина: {s.peek()}")	<i>20</i>
print(f"Размер: {len(s)}")	<i>2</i>
s.history()	

► **Результат:**

30

Вершина: 20

Размер: 2

История операций:

1. push(10)
2. push(20)
3. push(30)
4. pop() → 30

□ **Тест: Тема 14**

? Вопрос: Что такое self в методе класса?

- А) Ключевое слово Python
- Б) Ссылка на сам объект ← *правильный ответ*
- В) Имя класса
- Г) Тип данных

? Вопрос: Как называется метод, который вызывается при создании объекта?

- А) __create__
- Б) __new__
- В) __init__ ← *правильный ответ*
- Г) __start__

? Вопрос: Что делает super().__init__() в дочернем классе?

- А) Удаляет родительский класс
- Б) Вызывает конструктор родителя ← *правильный ответ*
- В) Создаёт копию объекта
- Г) Переопределяет все методы